Mid-Term Report: Query Scheduling Optimization in IVF-based Vector Search

Yujia Qian — yjqian19@mit.edu
Xiangyu Guan — xiang949@mit.edu

1. Project Status

Tasks	Complete	Pending
Core engine	custom IVF engine, MV + MM scan modes	—
Schedulers	Sequential, Batch(MV), Batch(MM)	optimization
Evaluation	parameter sweep, random vs. clustered comparison	clustered query construction, multi-run, sweep under clustered workload

Deliverables	Complete	Pending
A working system	all	—
Measured performance differences	First-round results on SIFT1M	Mentioned above in the evaluation part
Final report and video	—	all

2. System Overview

Engine

Our engine follows the IVF design from Faiss: base vectors are partitioned into 256 clusters via k-means, each cluster maintaining an inverted list. We use Faiss for centroid training but replace `index.search()` with a custom single-threaded implementation. Faiss's search is a stateless black box that processes each query independently, making cross-query optimizations invisible. Our engine exposes centroid lookup and list scanning as separate interfaces, giving the scheduler direct control over execution.

Search Methods

- **`search_one`** — single-query path: centroid lookup → scan 8 lists → top-k. Used by Sequential.
- **`search_batch_per_list`** — batch path with a list-major scan: iterates over inverted lists; for each list, computes distances for all queries that probe it before moving to the next. Each list is loaded from memory exactly once. Used by both batching schedulers, with two inner implementations:

Mode	Distance computation per list	When efficient
Batch(MV)	loop over queries, each a matrix × vector (`vecs @ q`)	small m, random queries
Batch(MM)	one matrix × matrix (`vecs @ Q.T`) for all m queries at once	large m, clustered queries

Batch(MM) amortizes memory bandwidth across all `m` queries sharing a list, but adds overhead when `m` is small.

Schedulers

The experiment compares three configurations:

- **Sequential** — `search_one`, one query at a time, no batching
- **Batch(MV)** — time-window batching + MV scan
- **Batch(MM)** — time-window batching + MM scan

Time-window batching uses a dual-trigger flush: whichever fires first between time limit Δt and maximum batch size MaxBS.

3. Experiment Setup

- **Dataset:** SIFT1M (1M × 128-d, L2 distance)
- **Device:** MacBook Pro, Apple M3 Pro, 18 GB RAM
- **Search parameters:** n_clusters=256, nprobe=8, k=10
- **Scheduler parameters:** Δt=5ms, MaxBS=128 fixed for comparison; separately swept as Δt ∈ {0.5, 1, 2, 5, 10, 20, 50} ms × MaxBS ∈ {32, 64, 128, 256} under random queries (full results recorded in a separate report)
- **Queries:** 10K queries from SIFT1M, arrival rate 2000 QPS (Poisson), tested under two workloads:
  - *Random* — queries sampled uniformly, probing largely disjoint inverted lists
  - *Clustered* — queries drawn from 10 spatial regions, with high centroid overlap across queries

  We test both because the number of queries sharing each inverted list (m) depends on workload structure — random is the worst case for batching, clustered is the best case.

4. Results

Random Queries

	Sequential	Batch(MV)	Batch(MM)
Recall@10	0.956	0.956	0.956

	Sequential	Batch(MV)	Batch(MM)
QPS	1793	**2212**	1721
Avg Latency (ms)	**0.6**	72.1	109.8
P95 Latency (ms)	**0.8**	103.4	135.7

Batch(MV) achieves the highest throughput (+23% over Sequential), at the cost of ×129 higher average latency due to queuing. Batch(MM) underperforms Sequential (−4% QPS) — on random queries, each list is shared by only ~4 queries on average, so GEMM adds matrix construction overhead without arithmetic intensity benefit, pushing average latency to ×197 over Sequential.

Clustered Queries

	Sequential	Batch(MV)	Batch(MM)
Recall@10	—	—	—
QPS	1896	2300	**2571**
Avg Latency (ms)	**0.5**	58.8	33.3
P95 Latency (ms)	**0.7**	104.1	63.4

Recall is not reported: clustered queries are sampled from base vectors, making self-retrieval recall undefined. With clustered queries, many queries share the same lists (large m), and Batch(MM) achieves the best throughput (+36% over Sequential). Notably, Batch(MM) also has lower latency than Batch(MV) — ×63 vs ×112 over Sequential — because GEMM's higher arithmetic intensity reduces scan time enough to more than offset queuing delay.

MV vs. MM: the role of m

The key variable is m — the number of queries sharing each inverted list per batch. On random queries, m is small and GEMM overhead dominates. On clustered queries, m is large and GEMM's O(m) arithmetic intensity pays off. This suggests an adaptive strategy: use MM only when m exceeds a threshold.

5. Potential Problems

1. **Clustered query workload construction is incomplete.** The current clustered queries are sampled directly from base vectors, so each query exists in the index and self-retrieval makes recall undefined. A more realistic construction — queries near but distinct from base vectors — remains to be built.

2. **Batch(MM) underperforms on random queries.** When m is small, GEMM overhead hurts rather than helps. An adaptive scan mode that switches between MV and MM based on observed m per batch would eliminate this regression.

3. **Results are based on a single run.** All numbers come from one run. Multi-run experiments (mean ± std) would improve statistical credibility.

4. **Parameter sweep was only conducted under random queries.** Running the same Δt × MaxBS sweep under clustered conditions may reveal different optimal parameters and a different MV vs. MM crossover point.

5. **Additional metrics could strengthen the argument.** Cache miss rate (`perf stat`) would directly validate the per-list reuse effect. Latency decomposition (queue delay vs. scan time) would show whether throughput gains come from better compute utilization or simply larger batches.

6. **Results are limited to a single-threaded setting.** The engine is single-threaded by design to isolate scheduling effects. On a multi-core CPU, queries can be parallelized across cores, which would reduce the relative benefit of batching. Whether the gains observed here survive in a multi-threaded setting remains to be evaluated.

6. Updated Timeline

The project is on schedule. Phase 1 work (Mar 27 → Apr 14) is complete as planned.

Phase	Dates	Tasks
Phase 1	Mar 27 → Apr 14 ✓	Core engine, schedulers, dataset, first round evaluation
Phase 2	Apr 14 → May 3	Optimize scheduling strategy, refine evaluation setup
Phase 3	May 3 → May 7	Compose report and prepare video

7. Division of Work

Member	So Far	Next
Xiangyu	schedulers, core engine improvement, evaluation	scheduler improvement, evaluation
Yujia	core engine base, evaluation, report	evaluation, report